

menler

Your turning point in the AI era.

AI APTITUDE QUESTION BANK

AI Agents and Workflows

About This Question Bank

A bank for practitioners building with agentic AI — covering agent architecture, tool use, prompt chaining, multi-agent coordination, and the safety considerations that matter in production.

5

Sets

15

Questions per Set

75

Total Questions

SET GUIDE

Set 1 · Agent Fundamentals

Agent fundamentals — what agents are and how they differ from basic LLM calls

Set 2 · Tool Use and Actions

Tool use and actions — how agents perceive, decide, and act through tools

Set 3 · Prompt Chaining

Prompt chaining — designing reliable multi-step pipelines

Set 4 · Multi-Agent Systems

Multi-agent systems — orchestration, delegation, and coordination patterns

Set 5 · Safety and Reliability

Safety and reliability — failure modes, guardrails, and responsible deployment

Each question has one correct answer. Select the best answer from the options provided.

Correct answers and explanations follow each question.

SET

1

Agent Fundamentals

Set 1 of 5 · 15 Questions · AI Agents and Workflows

Q1. What distinguishes an AI "agent" from a standard AI chatbot?

- A. An agent has higher intelligence and handles more complex questions.
- B. An agent takes multi-step actions autonomously using tools to achieve a goal.**
- C. An agent learns and improves from each conversation over time.
- D. An agent can access the internet and retrieve current information.

Correct Answer: B**Explanation**

An agent plans and executes multi-step sequences of actions — calling tools, using results to decide next steps, and iterating until a goal is achieved. A chatbot generates a single response. The key distinguishing feature is autonomous multi-step action, not intelligence, learning, or internet access per se.

Q2. What is the "observe-orient-decide-act" (OODA) loop relevant to in agentic AI?

- A. A debugging framework for identifying errors in multi-step AI pipelines.
- B. A compliance framework for monitoring AI agent behaviour in production.
- C. A training methodology for teaching agents to plan multi-step tasks.
- D. A mental model for how an AI agent processes perception and takes action iteratively.**

Correct Answer: D**Explanation**

The OODA loop (Observe — Orient — Decide — Act) is a useful mental model for agentic systems: the agent observes its environment (tool outputs, user input), orients (updates its understanding), decides the next action, and acts. This cycle repeats until the goal is achieved.

Q3. What is "tool use" (function calling) in the context of language model agents?

- A. The model can invoke predefined external functions and incorporate their results.**
- B. The model uses different internal parameters depending on the task type.
- C. The model selects from a library of pre-written prompt templates for each task.
- D. The model switches between text and code generation modes based on context.

Correct Answer: A

Explanation

Tool use (function calling) allows a language model to call external functions — APIs, code interpreters, search engines, databases — as part of generating a response. The model produces structured tool call outputs, the runtime executes them, and the results are fed back to the model.

Q4. What is the most important design constraint for an autonomous AI agent?

- A. Using the most capable available model as the agent's reasoning core.
- B. Minimising the number of steps the agent takes to complete a task.
- C. Clearly defined boundaries on what actions it is and is not permitted to take.**
- D. Ensuring the agent can retry failed steps without human intervention.

Correct Answer: C

Explanation

Action boundaries are the most critical design constraint. An agent without clear scope can take unintended, irreversible actions. The first design decision must be: what can this agent do autonomously, and what requires human approval? Model capability and efficiency are secondary to this safety-critical scope definition.

Q5. What does "planning" mean in an agentic AI system?

- A. The model predicting which future user requests are most likely to arrive.
- B. Decomposing a goal into a sequence of steps before beginning execution.**
- C. Scheduling agent jobs across multiple servers for parallel execution.
- D. Pre-generating several candidate responses before selecting the best one.

Correct Answer: B

Explanation

Planning is the process by which an agent takes a high-level goal and decomposes it into an ordered sequence of subtasks and actions. Good planning determines whether an agent can efficiently accomplish complex goals or wastes steps and gets stuck on dead ends.

Q6. What is the "ReAct" framework for AI agents?

- A. Interleaving reasoning and action steps — the model thinks, then acts, then thinks again.**
- B. A React.js framework for building user interfaces for AI agent products.
- C. A reinforcement learning approach for training agents through trial and error.
- D. A method for automatically generating tool definitions from API documentation.

Correct Answer: A

Explanation

ReAct (Reasoning + Acting) is a prompting framework where the model alternates between "Thought" steps (reasoning about the current state) and "Action" steps (calling a tool or producing output). This interleaving of reasoning and action improves agent reliability on complex tasks.

Q7. What is the most common failure mode of an AI agent in production?

- A. The agent uses too much compute and exceeds cost budgets unexpectedly.
- B. The agent refuses to take any action without explicit human confirmation.
- C. The agent produces outputs too slowly for real-time use cases.
- D. The agent gets stuck in loops or takes wrong actions when encountering unexpected inputs.**

Correct Answer: D

Explanation

Production agent failures most commonly involve handling unexpected inputs — edge cases that the agent's planning assumptions did not anticipate, causing it to loop, take incorrect actions, or get stuck in error states. Robust error handling and fallback logic are essential for production agents.

Q8. What is "scaffolding" in the context of AI agent development?

- A. A set of pre-written prompts that guide the agent through each task step.
- B. The process of gradually increasing task complexity during agent training.
- C. The code infrastructure that manages agent execution, tool calls, and state.**
- D. A technique for breaking large agent outputs into smaller manageable chunks.

Correct Answer: C

Explanation

Agent scaffolding is the surrounding code that drives the agent loop — parsing tool call outputs, routing results back to the model, managing state, handling errors, enforcing timeouts, and deciding when to stop. Good scaffolding is as important as good prompting for reliable agent behaviour.

Q9. What is the difference between a "single-step" and "multi-step" AI workflow?

- A. Single-step is one model call; multi-step chains multiple calls where each builds on the last.**
- B. Single-step handles simple tasks; multi-step requires a more capable model.
- C. Single-step uses one model; multi-step uses multiple different models.
- D. Single-step is synchronous; multi-step is always asynchronous.

Correct Answer: A

Explanation

Single-step workflows make one AI call and return the result. Multi-step workflows chain multiple calls — the output of each step becomes the input of the next. Multi-step is necessary for tasks too complex or long to handle in one context window, or where intermediate results inform later decisions.

Q10. What is "memory" in the context of an AI agent?

- A. The total amount of RAM available to the model during inference.
- B. The agent's ability to learn and update its weights from user interactions.
- C. The portion of the context window reserved for conversation history.
- D. Mechanisms for persisting and retrieving context across multiple agent steps or sessions.**

Correct Answer: D

Explanation

Agent memory refers to how agents store and access information beyond the immediate context window: in-context memory (within the current window), external memory (databases, vector stores), or episodic memory (conversation logs). Effective memory design determines whether agents can handle long tasks and learn from past interactions.

Q11. What makes "irreversible actions" especially important to handle carefully in agents?

- A. Irreversible actions take longer and increase per-step latency.
- B. Mistakes cannot be undone without significant effort or harm.**
- C. Irreversible actions require more compute than reversible ones.
- D. Regulatory frameworks prohibit agents from taking irreversible actions without a license.

Correct Answer: B

Explanation

Irreversible actions — sending emails, posting content, deleting files, spending money — cannot be undone if the agent makes an error. This makes human review before execution critical. The consequences of an irreversible mistake compound the importance of getting these actions right the first time.

Q12. What is "chain of thought" most useful for in an agentic context?

- A. Linking multiple agent instances together in a sequential pipeline.
- B. Tracking the sequence of tool calls an agent has made for debugging.
- C. Having the agent reason explicitly before deciding which action to take next.**
- D. Generating a narrative explanation of the agent's actions for the user.

Correct Answer: C

Explanation

In agents, chain of thought (or "thinking" steps) allows the model to reason about the current state, consider options, and deliberate before committing to an action. This reduces impulsive errors and improves the quality of action selection, especially for complex multi-step tasks.

Q13. What is the primary purpose of a system prompt in an AI agent?

- A. Providing the agent with real-time data it needs for the current task.
- B. Setting the agent's output format for all responses it generates.
- C. Storing the conversation history so the agent can reference previous turns.
- D. Defining the agent's role, capabilities, constraints, and available tools.**

Correct Answer: D

Explanation

The system prompt is the persistent instruction set that defines who the agent is, what tools it has access to, what it is allowed to do, what its goal is, and what constraints it must operate within. It is the foundational configuration of any agentic system.

Q14. What does "agent trajectory" refer to?

- A. The latency curve of an agent across multiple task runs.
- B. The sequence of observations, reasoning steps, and actions an agent takes to complete a task.**
- C. The path an agent takes through a decision tree during planning.
- D. The history of user prompts that led to the current agent invocation.

Correct Answer: B

Explanation

An agent trajectory is the full record of what happened during a task run: what the agent observed, how it reasoned, what actions it took, what results it received, and how it responded to those results. Trajectories are essential for debugging, evaluation, and improving agent behaviour.

Q15. What is "grounding" in the context of AI agents?

- A. Anchoring agent actions and beliefs to verified external information sources.**
- B. Connecting the agent to a physical hardware environment for real-world tasks.
- C. Providing the agent with explicit instructions at the start of each session.
- D. Limiting the agent's output to text that can be directly executed as code.

Correct Answer: A

Explanation

Grounding means ensuring the agent's beliefs and claims are anchored to external, verifiable sources rather than generated from internal patterns alone. Grounded agents retrieve information from trusted databases or documents before acting on it, reducing hallucination-driven errors.

SET

2

Tool Use and Actions

Set 2 of 5 · 15 Questions · AI Agents and Workflows

Q16. When a language model "calls a function," what actually happens?

- A. The model directly executes code within its own inference process.
- B. The model retrieves a pre-computed result from a cached function output store.
- C. The model outputs a structured request; external code executes it and returns the result.**
- D. The model generates pseudocode that a human developer then implements.

Correct Answer: C**Explanation**

Function calling is a structured output pattern. The model generates a JSON-formatted call specification; the surrounding scaffolding detects this, calls the actual external function, and feeds the result back to the model as a new message. The model itself never executes code directly.

Q17. What information must a tool definition include for Claude to call it effectively?

- A. Tool name, description of what it does, and parameter schema with types and descriptions.**
- B. Tool name, the API endpoint URL, and authentication credentials.
- C. Tool name, the programming language it is implemented in, and expected runtime.
- D. Tool name, a list of example inputs, and the expected output format.

Correct Answer: A**Explanation**

Effective tool definitions require a clear name, a precise description (so the model understands when and why to call it), and a parameter schema specifying each parameter's name, type, and description. Without clear descriptions, models call the wrong tool or pass incorrect parameters.

Q18. What is "parallel tool calling" and when is it useful?

- A. Running the same tool multiple times with different inputs to compare results.
- B. Using multiple AI models in parallel, each with access to different tools.
- C. Pre-warming tool execution environments before the agent begins a task.
- D. Invoking multiple tools simultaneously when results are independent of each other.**

Correct Answer: D

Explanation

Parallel tool calling allows the model to invoke multiple tools in the same step when their results are independent. For example, searching for information in three different sources simultaneously rather than sequentially reduces total latency significantly.

Q19. What is the most important consideration when designing a tool for an AI agent?

- A. Minimising the tool's response time to reduce agent latency.
- B. Clear, unambiguous description of when and how to call it to prevent misuse.**
- C. Ensuring the tool can handle any possible input without throwing exceptions.
- D. Matching the tool's interface to standard REST API conventions.

Correct Answer: B

Explanation

The model decides when and how to call a tool based entirely on its description. Ambiguous descriptions lead to wrong tool selection or incorrect parameter usage. The tool description is a contract between the tool developer and the model — precision here is more important than performance or interface conventions.

Q20. You give an agent a web search tool. What is the most important risk to manage?

- A. Web search may return results that exceed the agent's context window.
- B. The agent may make too many search calls and exceed API rate limits.
- C. The agent may act on false or malicious information retrieved from the web.**
- D. Web search results may be in a language the model cannot process.

Correct Answer: C

Explanation

An agent that trusts and acts on retrieved web content is vulnerable to misinformation, out-of-date information, and prompt injection (malicious instructions embedded in web pages). The primary risk is acting on unverified or malicious retrieved content, not rate limits or language issues.

Q21. What is "prompt injection" in an agentic context?

- A. Injecting additional instructions into the agent's system prompt at runtime.
- B. A technique for inserting few-shot examples into the agent's prompt dynamically.
- C. An attack where an adversary sends the agent an unusually long input to cause errors.
- D. Malicious instructions embedded in content the agent reads that hijack its behaviour.**

Correct Answer: D

Explanation

Prompt injection in agents occurs when content retrieved from external sources (web pages, emails, documents) contains text that looks like instructions to the model, causing it to deviate from its original task. This is a critical security concern for agents that process untrusted external content.

Q22. What is the correct handling when a tool call returns an error?

- A. The agent should detect the error, decide whether to retry or use a fallback, and proceed accordingly.**
- B. The agent should immediately terminate and report failure to the user.
- C. The agent should retry the same call indefinitely until it succeeds.
- D. The agent should proceed as if the tool returned an empty successful response.

Correct Answer: A

Explanation

Robust agents detect tool errors and reason about the appropriate response: retry if the error is transient, try an alternative approach if the error is persistent, or escalate to the user if recovery is impossible. Blind termination, infinite retries, and ignoring errors all produce worse outcomes.

Q23. You are designing an agent that can send emails on the user's behalf. What is the non-negotiable safeguard?

- A. Limiting the agent to sending emails only to addresses in the user's contacts.
- B. Human review and confirmation before any email is sent.**
- C. Adding a standard AI disclosure footer to every email the agent sends.
- D. Logging all sent emails to a database for audit purposes.

Correct Answer: B

Explanation

Sending email is an irreversible external action with real consequences. Human confirmation before sending is the non-negotiable safeguard — no other control (contact list restrictions, logging, disclosure) compensates for the risk of sending wrong emails without human approval.

Q24. What is the difference between a "deterministic" and "non-deterministic" tool in agent design?

- A. Deterministic tools execute instantly; non-deterministic tools have variable latency.
- B. Deterministic tools are safe for agents to call; non-deterministic ones require human approval.
- C. Deterministic tools are built in-house; non-deterministic tools call third-party APIs.
- D. Deterministic tools always return the same output for the same input; non-deterministic tools do not.**

Correct Answer: D

Explanation

Deterministic tools (calculators, data lookups by ID) return consistent results for given inputs. Non-deterministic tools (web search, current time, randomness sources) return different results on different calls. This distinction matters for agent reliability and debugging — non-deterministic tool results make trajectories harder to reproduce.

Q25. What does "tool result grounding" mean in a RAG-enabled agent?

- A. The tool results are stored in a permanent database for future reference.
- B. The agent verifies tool results by calling a second verification tool.
- C. The agent's answer is based on retrieved document content rather than training knowledge.**
- D. The tool results are formatted and grounded to a specific output template.

Correct Answer: C

Explanation

Tool result grounding means the agent's responses are explicitly based on the content returned by retrieval or tool calls — not the model's training knowledge. This is the primary mechanism by which RAG reduces hallucination: the model is instructed to cite and reason from retrieved documents, not generate from memory.

Q26. What is the most reliable way to prevent an agent from calling a sensitive tool inappropriately?

- A. Adding a human approval step before every tool call the agent makes.
- B. Clear tool descriptions specifying exactly when the tool should and should not be called.**
- C. Removing sensitive tools from the agent's tool list unless explicitly needed.
- D. Using a safety classifier to intercept and block inappropriate tool calls.

Correct Answer: B

Explanation

The most reliable primary prevention is clear tool descriptions — telling the model explicitly what situations warrant calling the tool and what situations do not. Removing the tool when not needed is a strong secondary control. Safety classifiers add a layer but are not the primary mechanism.

Q27. What is "structured output" and why is it important for tool use?

- A. Machine-parseable formatted output (like JSON) that the scaffolding code can reliably process.**
- B. Formatted text output that is presented to users in a visually organised way.
- C. Output that is generated according to a fixed response template.
- D. Output that has been reviewed and approved by a human before delivery.

Correct Answer: A

Explanation

Structured outputs (JSON, XML) are essential for tool-use scaffolding because the surrounding code needs to parse the model's tool call specification to execute it. If the model generates malformed or unstructured tool calls, the scaffolding fails. Reliable structured output is foundational to reliable agent systems.

Q28. You notice your agent is calling the same tool repeatedly with the same parameters. The most likely cause is:

- A. The tool has a bug that returns incorrect results, confusing the agent.
- B. The agent's context window is too small to remember it already called the tool.
- C. The agent is stuck in a loop because the tool result is not resolving its uncertainty.**
- D. The system prompt instructed the agent to verify all tool results by re-calling.

Correct Answer: C

Explanation

Repeated identical tool calls indicate the agent is in a loop — it expected the tool result to change its decision state, but it didn't. This usually happens when the tool result doesn't contain what the agent needs to progress, or when the agent's plan doesn't account for tool failure.

Q29. What is the most important metric to track for tool reliability in a production agent?

- A. Average tool response latency across all calls in production.
- B. Total number of tool calls made per user session.
- C. The percentage of tool calls that return non-empty results.
- D. Tool call error rate and the agent's recovery success rate when errors occur.**

Correct Answer: D

Explanation

Tool error rate and recovery success rate are the critical reliability metrics. A tool that fails frequently and which the agent cannot recover from degrades the entire system. Latency, call volume, and non-empty result rate are operational metrics; they don't capture whether the agent is actually succeeding.

Q30. What is the correct approach when an agent needs a tool that does not yet exist?

- A. Use the most general-purpose tool available as a substitute, even if it does more.
- B. Build a minimal, well-documented tool that does exactly what is needed and nothing more.**
- C. Ask the model to simulate the tool's output using its training knowledge.
- D. Build a tool that handles all possible future needs to avoid rebuilding later.

Correct Answer: B

Explanation

The correct approach is building minimal, focused tools with clear, precise interfaces. Over-general tools introduce misuse risk; simulated tool outputs introduce hallucination; building for future needs increases complexity unnecessarily. Start with the minimum tool that does exactly what is needed.

SET

3

Prompt Chaining

Set 3 of 5 · 15 Questions · AI Agents and Workflows

Q31. What is "prompt chaining" and when does it produce better results than a single prompt?

- A. Sequential prompts where each output becomes the next input — better for complex, multi-stage tasks.**
- B. Linking multiple AI models together where each specialises in a different modality.
- C. Repeating the same prompt multiple times and selecting the best output.
- D. Appending follow-up instructions to a prompt to extend a single response.

Correct Answer: A

Explanation

Prompt chaining decomposes complex tasks into stages, where each step has a focused, well-defined scope. This is more reliable than trying to accomplish everything in one prompt because each step can be validated, corrected, or re-run independently, and the model has more focused context at each stage.

Q32. What is the main reliability advantage of prompt chaining over a single long prompt?

- A. Chaining is faster because each individual prompt is shorter.
- B. Chaining uses less total compute than a single long prompt.
- C. Each step can be verified and corrected before it propagates errors to the next step.**
- D. Chaining allows different models to be used for different steps.

Correct Answer: C

Explanation

The primary reliability advantage is error containment. In a single long prompt, early errors cascade through the entire output. In a chain, each step's output can be validated before it becomes the input for the next step, and failed steps can be retried without restarting the whole task.

Q33. What is a "map-reduce" pattern in prompt chaining?

- A. Process large documents by dividing them across multiple models simultaneously.
- B. Apply the same operation to many items in parallel (map), then aggregate results (reduce).**
- C. Generate multiple candidate outputs and select the one that reduces perplexity.
- D. A data engineering pattern for preprocessing training data before model ingestion.

Correct Answer: B

Explanation

Map-reduce in prompt chaining processes many items independently in parallel (map) then combines results into a final output (reduce). For example: summarise 50 documents individually (map), then synthesise those summaries into a single executive briefing (reduce).

Q34. What is the most important design principle for individual steps in a prompt chain?

- A. Each step should be as long as possible to minimise the number of steps.
- B. Each step should produce a self-contained final output that needs no further processing.
- C. Each step should include instructions for handling all possible error conditions.
- D. Each step should have a single, clearly defined input and output.**

Correct Answer: D

Explanation

Single responsibility is the most important principle for chain steps: one input, one clearly defined task, one well-defined output. Steps with multiple responsibilities are harder to validate, debug, and retry, and tend to produce less reliable results.

Q35. You are building a chain that extracts information from documents and writes a report. The most important step to include between extraction and writing is:

- A. A validation step that checks the extracted information for completeness and accuracy.**
- B. A formatting step that converts extracted data to the report's visual template.
- C. A planning step where the model outlines the report structure.
- D. A length-checking step to ensure the report will fit within the context window.

Correct Answer: A

Explanation

Validating extracted information before using it to write the report prevents extraction errors from propagating into the final output. Catching errors at the extraction stage is far cheaper than rewriting a full report because the extraction was wrong.

Q36. What is "dynamic chain construction" in an AI workflow?

- A. The chain is built by sampling randomly from a library of available prompt steps.
- B. The agent determines which steps to execute based on intermediate results.**
- C. New chain steps are added by the user as the task progresses.
- D. The model dynamically selects which language model to call at each step.

Correct Answer: B

Explanation

Dynamic chain construction means the workflow path is not fixed in advance — the agent inspects intermediate results and decides which steps to take next. This enables adaptive workflows that handle branching logic, conditional execution, and dynamic task discovery.

Q37. What is the most common failure mode when chaining prompts?

- A. The total token count across the chain exceeds what is cost-effective.
- B. Different steps use different prompt formats that produce incompatible outputs.
- C. The final step loses coherence because the context window has been exceeded.
- D. Errors in early steps compound and corrupt all subsequent steps.**

Correct Answer: D

Explanation

Error propagation is the most common and damaging failure mode. If step 1 extracts wrong information, every step that follows operates on that wrong information, producing plausible-looking but fundamentally incorrect final outputs. Validation gates between steps are the primary mitigation.

Q38. How should you handle a step in a prompt chain that fails or produces bad output?

- A. Restart the entire chain from step one with a better initial prompt.
- B. Pass the failed output to the next step and flag it as uncertain.
- C. Detect the failure, retry with a modified prompt or different approach, then continue.**
- D. Ask the user to provide the correct output for the failed step manually.

Correct Answer: C

Explanation

Robust chains detect step failures and retry with modifications — adjusting the prompt, trying a different strategy, or using a fallback — before proceeding. Restarting from step one is wasteful. Passing bad output forward corrupts later steps. User manual input should be the last resort, not the default.

Q39. What is "prompt compression" and why does it matter in long chains?

- A. Reducing the length of each chain step's prompt to speed up generation.
- B. Summarising intermediate outputs to prevent context accumulation from filling the window.**
- C. Removing redundant instructions from prompts to reduce token costs.
- D. A technique for combining multiple chain steps into one to reduce API calls.

Correct Answer: B

Explanation

As chain outputs accumulate in context, the effective context window fills up. Prompt compression — summarising or extracting only the necessary information from intermediate outputs — prevents the model from losing earlier context or having to handle an inefficiently long prompt at later steps.

Q40. What distinguishes a "deterministic chain" from a "branching chain"?

- A. A deterministic chain always follows the same steps; a branching chain routes based on intermediate outputs.**
- B. A deterministic chain uses the same model at each step; a branching chain uses different models.
- C. A deterministic chain is synchronous; a branching chain is asynchronous.
- D. A deterministic chain is simpler to build; a branching chain requires more compute.

Correct Answer: A

Explanation

A deterministic chain executes the same sequence of steps every time. A branching chain inspects intermediate outputs and conditionally routes to different next steps based on the results. Branching chains are more powerful but harder to test exhaustively because different paths may rarely be executed.

Q41. What is "output parsing" in prompt chaining and why can it fail?

- A. Translating the final output into the user's preferred language.
- B. Running grammar and spell-check on model outputs before they move to the next step.
- C. Extracting structured data from a model's text output — fails when format is inconsistent.**
- D. Converting model outputs to speech for audio applications.

Correct Answer: C

Explanation

Output parsing extracts structured information (e.g., JSON fields, specific data points) from model-generated text. It fails when the model generates output that does not match the expected format — wrong field names, missing fields, malformed JSON, or free text where structured data was expected.

Q42. When is it appropriate to use a single complex prompt versus a chain of simpler prompts?

- A. Single prompts are always better because they require fewer API calls.
- B. Chains are always better because each step can be individually optimised.
- C. Single prompts are better for speed; chains are better when accuracy is critical.
- D. Single prompts work for straightforward tasks; chains are better for complex multi-stage workflows.**

Correct Answer: D

Explanation

Single prompts are appropriate for tasks that can be done well in one pass. Chains are better for tasks that have distinct stages (extract, validate, synthesise), where intermediate results must be checked, or where the total task is too large for one context window. Neither is universally superior.

Q43. What is the most important thing to log in a production prompt chain?

- A. The total token count and cost of each chain execution.
- B. Every step's inputs, outputs, and any errors, for debugging and quality monitoring.**
- C. Only the final output and whether the chain succeeded or failed.
- D. The model version used at each step for reproducibility tracking.

Correct Answer: B

Explanation

Full step-level logging — inputs, outputs, and errors at every step — is essential for debugging, quality monitoring, and root cause analysis when chains fail or produce wrong results. Cost tracking, final-output logging, and model version tracking are all valuable but secondary to this diagnostic capability.

Q44. A prompt chain produces a good result 90% of the time but fails 10%. The best engineering response is:

- A. Accept the 10% failure rate and notify users when failures occur.
- B. Switch to a more capable model to reduce the failure rate.
- C. Add validation steps and error handling to catch and recover from the 10% failures.**
- D. Redesign the chain so it only runs on the 90% of inputs where it is expected to succeed.

Correct Answer: C

Explanation

A 10% failure rate in production means 1 in 10 tasks fails, which is typically unacceptable. The correct response is engineering — adding validation steps to catch failures, error handling to attempt recovery, and fallback paths. These controls are more reliable than model capability upgrades alone.

Q45. What is a "handoff" in a prompt chain context?

- A. Passing the output of one step as the structured input to the next step.**
- B. Transferring a user session from an AI agent to a human support agent.
- C. Moving a workflow from development to production deployment.
- D. Shifting responsibility for a chain from one developer to another.

Correct Answer: A

Explanation

A handoff is the structured transfer of information between chain steps — the output format of step N must match the expected input format of step N+1. Well-designed handoffs have clear contracts (data types, field names, formats), making chains reliable and modular.

SET

4

Multi-Agent Systems

Set 4 of 5 · 15 Questions · AI Agents and Workflows

Q46. What is a "multi-agent system" in AI?

- A. A single AI model that simulates multiple personas in a conversation.
- B. A cluster of servers running the same AI model for high availability.
- C. An AI system that can call multiple external APIs in a single step.
- D. Multiple AI agents working together, each handling a part of a complex task.**

Correct Answer: D**Explanation**

A multi-agent system deploys multiple specialised agents that collaborate — each agent may have different tools, instructions, or capabilities, and they pass work and results between them. This allows parallelisation of complex tasks and specialisation of roles within a larger workflow.

Q47. What is the primary advantage of using specialised agents versus one general-purpose agent for complex tasks?

- A. Specialised agents use less compute than a single general-purpose agent.
- B. Specialised agents can be independently optimised and tested for their specific role.**
- C. Specialised agents can use different underlying models for different subtasks.
- D. Specialised agents are easier to prompt and require shorter system prompts.

Correct Answer: B**Explanation**

Specialisation allows each agent to be designed, prompted, and evaluated for its specific role — a research agent, a writing agent, and a verification agent each have focused, optimised configurations. A single general-purpose agent must handle all roles with a shared, less focused context.

Q48. What is an "orchestrator" agent?

- A. An agent that manages the infrastructure and compute resources for other agents.
- B. An agent that translates between different agents' output formats.
- C. An agent that coordinates other agents — assigning tasks and synthesising their results.**
- D. The most capable agent in a system that overrides other agents' decisions.

Correct Answer: C

Explanation

An orchestrator agent manages the workflow — it receives the user goal, decomposes it into subtasks, assigns subtasks to specialised worker agents, monitors their progress, synthesises their outputs, and determines when the overall goal has been achieved.

Q49. What is the biggest coordination challenge in multi-agent systems?

- A. Ensuring agents share consistent context and don't operate on contradictory information.**
- B. Preventing agents from using too many tokens in aggregate.
- C. Ensuring each agent uses the same AI model version.
- D. Managing API rate limits when multiple agents call external tools simultaneously.

Correct Answer: A

Explanation

Context consistency is the core coordination challenge. If Agent A makes a decision based on certain information, Agent B (working in parallel) must have access to the same information and any updates from Agent A, or their work will be based on divergent assumptions, producing contradictory or conflicting outputs.

Q50. What is "agent role confusion" and how does it manifest?

- A. The user cannot tell which agent in a system produced a given output.
- B. An agent fails to understand what role the user wants it to play.
- C. Multiple agents produce identical outputs because they have similar role definitions.
- D. An agent acts outside its assigned role, taking actions intended for another agent.**

Correct Answer: D

Explanation

Role confusion occurs when an agent ignores its assigned scope and attempts tasks outside its mandate — a research agent that tries to write the final report, or a writer agent that modifies the research brief. Clear, narrow role definitions and scope constraints prevent this.

Q51. What is "emergent behaviour" in multi-agent systems and why is it a risk?

- A. Agents developing new capabilities through interaction with each other.
- B. Agents that spontaneously develop efficient workflows without being explicitly programmed.
- C. Unexpected system-level behaviour arising from agent interactions not anticipated by designers.**
- D. A single agent discovering a novel approach that improves the whole system's performance.

Correct Answer: C

Explanation

Emergent behaviour is system-level outcomes that arise from agent interactions that designers did not anticipate or intend. In AI systems, this can manifest as agents coordinating to take actions or reach conclusions that no individual agent was designed to do — some of which may be harmful or unintended.

Q52. What is a "deadlock" in a multi-agent system?

- A. Two or more agents each waiting for the other to complete before they can proceed.**
- B. A situation where all agents produce identical outputs and the system cannot select one.
- C. An agent that keeps sending requests to a tool that is unavailable.
- D. A failure mode where an orchestrator loses track of which agents are active.

Correct Answer: A

Explanation

A deadlock occurs when agents are in a mutual wait state — Agent A is waiting for Agent B's output to proceed, while Agent B is waiting for Agent A's output. Without external intervention or a timeout mechanism, the system halts indefinitely.

Q53. What is the most appropriate use of parallel agent execution?

- A. All tasks in a multi-agent system, to minimise total completion time.
- B. Tasks that are independent of each other and whose results can be combined afterward.**
- C. Tasks where an earlier agent's output is uncertain and needs verification.
- D. Tasks where the final result needs to be produced as quickly as possible.

Correct Answer: B

Explanation

Parallelism is appropriate only for genuinely independent subtasks — where Agent A's execution does not depend on Agent B's output. Forcing parallelism on dependent tasks causes coordination failures. Independent parallel tasks (e.g., researching five different topics simultaneously) are the correct use case.

Q54. What is the correct approach when one agent in a multi-agent system produces an error?

- A. The entire multi-agent workflow restarts from the beginning.
- B. The failed agent retries indefinitely until it succeeds.
- C. The orchestrator detects the error and decides whether to retry, use a fallback, or escalate.**
- D. The orchestrator skips the failed agent's task and proceeds without its output.

Correct Answer: C

Explanation

The orchestrator is responsible for error handling in multi-agent systems — detecting failures from worker agents and making intelligent recovery decisions: retry the same agent, substitute a different agent, use cached results from a previous run, or escalate to a human if recovery is impossible.

Q55. What is "agent communication protocol" and why does it matter?

- A. The rate at which agents send messages to each other during execution.
- B. The security protocol used to authenticate agent-to-agent API calls.
- C. The programming language used to implement each agent's reasoning logic.
- D. The structured format by which agents exchange information — critical for interoperability.**

Correct Answer: D

Explanation

A communication protocol defines how agents share information — what format messages take, what fields are required, and how to interpret different message types. Without a clear protocol, agents produce outputs in incompatible formats that other agents cannot reliably parse and act on.

Q56. What distinguishes "cooperative" from "competitive" multi-agent architectures?

- A. Cooperative agents work toward a shared goal; competitive agents optimise different objectives.**
- B. Cooperative agents share compute resources; competitive agents have isolated resources.
- C. Cooperative agents can call each other's tools; competitive agents cannot.
- D. Cooperative agents are designed by the same team; competitive ones are from different vendors.

Correct Answer: A

Explanation

In cooperative multi-agent systems, all agents are aligned on a shared goal and optimise for collective success. In competitive architectures (debate, red team vs blue team), agents optimise different or opposing objectives. The choice of architecture fundamentally changes system design and trust requirements.

Q57. What is a "critic agent" and when is it useful?

- A. An agent that monitors system performance and alerts when metrics degrade.
- B. An agent that evaluates another agent's output and identifies errors or improvements.**
- C. An agent that filters harmful content from other agents' outputs.
- D. An agent that rejects tasks outside its scope and returns them to the orchestrator.

Correct Answer: B

Explanation

A critic agent independently evaluates another agent's output — checking for errors, inconsistencies, logical flaws, or quality issues. It is most useful in high-stakes workflows where one agent's unchecked output would be used directly in a consequential decision or downstream process.

Q58. What is the most important human oversight requirement for multi-agent systems?

- A. A human reviewing every message that passes between agents.
- B. A human approving the addition of each new agent to the system.
- C. Daily audits of all agent logs by a designated system administrator.
- D. Checkpoint reviews at consequential decision points before irreversible actions.**

Correct Answer: D

Explanation

Checkpoints at consequential decision points — moments where the agent system is about to take an irreversible or high-stakes action — are the most important oversight mechanism. Reviewing every inter-agent message is impractical; daily audits catch problems too late; agent approval gatekeeping is not the right control.

Q59. What is "agent handoff" in a customer service context?

- A. An AI agent transferring a conversation to a human agent when it cannot resolve the issue.**
- B. One AI agent passing a task to a specialised AI agent with more relevant tools.
- C. A system that routes new incoming queries to the most available agent.
- D. The process of switching a user from one product's AI to another product's AI.

Correct Answer: A

Explanation

In customer service, agent handoff is the critical transition from AI to human handling. A well-designed handoff includes context transfer (conversation history, attempted solutions, customer sentiment) and a smooth user experience. Poor handoffs that lose context are a primary source of customer frustration in AI-assisted support.

Q60. What is the most reliable way to test a multi-agent system before production deployment?

- A. Unit testing each agent in isolation with simulated inputs from other agents.
- B. Testing the orchestrator in isolation with all worker agents mocked.
- C. Running end-to-end tests on realistic scenarios that cover the main task paths and edge cases.**
- D. Deploying to a canary user group and measuring error rates in production.

Correct Answer: C

Explanation

Multi-agent systems must be tested end-to-end on realistic scenarios because system-level behaviour emerges from agent interactions, not from individual agents alone. Unit tests catch individual agent errors; they don't catch coordination failures, context drift, or emergent behaviours that only appear in full-system execution.

SET

5

Safety and Reliability

Set 5 of 5 · 15 Questions · AI Agents and Workflows

Q61. What is the "minimal footprint" principle in agent design?

- A. Request only the permissions and tools needed for the specific task.**
- B. Use the smallest possible model that can complete the task adequately.
- C. Minimise the number of steps the agent takes to complete each task.
- D. Limit the agent to read-only tool access to prevent unintended changes.

Correct Answer: A**Explanation**

Minimal footprint means agents should request only the permissions they need, retain only the state they need, and access only the data they need. Unnecessarily broad tool access and permissions increase the blast radius of mistakes and security vulnerabilities.

Q62. What is "reward hacking" as it applies to AI agent systems?

- A. A security vulnerability where an attacker manipulates the agent's reward signal.
- B. An agent that requests rewards (credits, tokens) from users in exchange for help.
- C. A training failure where the reward model assigns high scores to clearly wrong outputs.
- D. An agent achieves its specified objective by means that violate the designer's intent.**

Correct Answer: D**Explanation**

Reward hacking (specification gaming) occurs when an agent finds a way to satisfy its specified goal metric without achieving the underlying intent. For example, an agent told to "maximise user engagement" might keep users stuck in loops rather than actually helping them. The measure is optimised; the purpose is subverted.

Q63. What is the most important difference between "safe" and "unsafe" agentic behaviour?

- A. Safe agents are slower because they verify each step; unsafe agents are faster.
- B. Safe agents stay within their authorised scope; unsafe agents take unauthorised actions.**
- C. Safe agents use less capable models; unsafe agents use the most powerful model.
- D. Safe agents have human oversight; unsafe agents operate in a production sandbox.

Correct Answer: B

Explanation

Safe agentic behaviour is defined by operating within authorised scope — doing what was sanctioned by the principal hierarchy (user and operator), nothing more. Unsafe behaviour is taking actions that exceed the given authority, regardless of whether those actions seem beneficial.

Q64. What is "cascading failure" in an agent system?

- A. When multiple agents fail at the same time due to a shared resource outage.
- B. A failure mode where an agent's output keeps growing until it exceeds context limits.
- C. An error in one component propagates and amplifies through dependent components.**
- D. When retry logic causes exponentially increasing API calls that overwhelm a service.

Correct Answer: C

Explanation

Cascading failure in agents occurs when an error in one step or agent corrupts the inputs of downstream steps or agents, which then produce their own errors, propagating the failure through the system. Without error detection and circuit-breaking at each handoff, one early error can corrupt an entire complex workflow.

Q65. What is the most effective way to prevent an agent from taking an action with irreversible consequences?

- A. Require explicit human confirmation before the agent executes any irreversible action.**
- B. Add a post-execution check that undoes any action the agent should not have taken.
- C. Train the agent to recognise irreversible actions and automatically refuse them.
- D. Limit the agent's tool access so it cannot access any tool with irreversible effects.

Correct Answer: A

Explanation

Pre-execution human confirmation is the only reliable prevention for irreversible actions. Post-execution checks cannot undo what is done. Training-based refusal can be circumvented or fail on edge cases. Blanket tool removal is too restrictive for agents that legitimately need to take some consequential actions.

Q66. What is "scope creep" in agent design and why is it dangerous?

- A. The system prompt gradually grows too long for the context window.
- B. The agent's tool list expands over time without systematic review.
- C. The agent gradually takes on tasks beyond its intended scope, increasing risk.**
- D. The agent's performance degrades as the deployment scope expands.

Correct Answer: C

Explanation

Scope creep occurs when an agent begins handling tasks outside its designed boundaries — often because it seems capable and users start asking for more. This is dangerous because safety testing, permissions, and oversight were designed for the original scope, not the expanded one.

Q67. What is the most important log to maintain for agent safety auditing?

- A. A log of every user prompt that triggered an agent execution.
- B. A record of every API call and its cost for billing and performance analysis.
- C. A log of every tool error and the agent's recovery strategy.
- D. A complete record of every action taken and its outcome, with timestamps.**

Correct Answer: D

Explanation

A complete action log — what the agent did, when, and what resulted — is the foundational record for safety auditing. It allows investigators to reconstruct exactly what happened when something goes wrong. Prompt logs, cost logs, and error logs are all subsets of or supplements to this primary audit record.

Q68. What is "principal hierarchy" in the context of AI agent safety?

- A. The order in which agents execute in a multi-agent pipeline.
- B. The authority structure defining whose instructions the agent should prioritise.**
- C. The chain of command within the development team responsible for the agent.
- D. The ranking of tools the agent should prefer when multiple tools could serve a purpose.

Correct Answer: B

Explanation

Principal hierarchy defines whose instructions the agent should follow and in what priority order. For Claude, this is Anthropic (through training) > operator (through system prompt) > user (through conversation). Understanding this hierarchy is essential for designing agents that follow appropriate instructions and resist manipulation.

Q69. What is the correct response when an agent is given an instruction that conflicts with its safety constraints?

- A. Follow the instruction since the user or operator has authority over the agent.
- B. Attempt to partially follow the instruction while minimising safety violations.
- C. Follow the safety constraints and explain why the conflicting instruction cannot be followed.**
- D. Escalate the conflict to the operator for resolution before proceeding.

Correct Answer: C

Explanation

Safety constraints are non-negotiable and take precedence over user or operator instructions. When an instruction conflicts with safety constraints, the agent should decline to follow the unsafe instruction and explain why. Instructions can narrow what an agent does within safety bounds; they cannot override safety bounds.

Q70. Why is "rate limiting" important in production AI agent systems?

- A. Prevents runaway agents from making unbounded API calls that cause cost or service failures.**
- B. Ensures fair distribution of compute resources between multiple concurrent users.
- C. Prevents agents from generating outputs that exceed the context window size.
- D. Limits the number of agents that can be deployed simultaneously in the system.

Correct Answer: A

Explanation

Without rate limits, a malfunctioning or looping agent can make thousands of API calls in seconds, exhausting budgets, hitting external API rate limits, and causing service disruptions. Rate limiting is a critical circuit-breaker that prevents runaway agent behaviour from causing cascading operational failures.

Q71. What is the most important test to run before deploying an AI agent to production?

- A. Performance testing — does the agent complete tasks within the SLA time limits?
- B. Adversarial testing — can the agent be prompted to take actions outside its intended scope?**
- C. Cost testing — does the agent stay within the target token budget per task?
- D. Compatibility testing — does the agent work correctly across all supported platforms?

Correct Answer: B

Explanation

Adversarial testing is the most critical pre-deployment test for agents. Agents in production will encounter manipulative users, unexpected inputs, and edge cases. Understanding where the agent's scope boundaries can be pushed or circumvented is essential before exposing it to real users.

Q72. What does "fail-safe" mean in an agent system design?

- A. The agent automatically retries all failed operations until they succeed.
- B. The agent stores its state so it can resume from the last successful step after a failure.
- C. A backup agent that activates when the primary agent fails.
- D. When the system fails, it defaults to a safe state rather than an unsafe one.**

Correct Answer: D

Explanation

Fail-safe design means that when something goes wrong — tool failure, context overflow, unexpected input — the system's default response is conservative (do nothing, ask the human, stop and report) rather than attempting to proceed with potentially wrong or dangerous actions.

Q73. What is "human-in-the-loop" most critical for in agent design?

- A. All agent tasks, since agents should never operate without human supervision.
- B. The initial task specification, to ensure the agent has the correct goal.
- C. High-stakes, irreversible, or uncertain decisions where agent errors have significant consequences.**
- D. Error recovery steps, where the agent cannot determine the correct next action.

Correct Answer: C

Explanation

Human-in-the-loop is most critical at high-stakes decision points — where the agent is about to take an action with significant, irreversible, or hard-to-detect consequences. Requiring human approval for every step is impractical; applying it selectively to critical junctures is the appropriate design.

Q74. What is the most reliable indicator that an AI agent system needs a human escalation path?

- A. The agent has been running for longer than its typical task completion time.
- B. The agent encounters a situation where proceeding would require taking an irreversible action under uncertainty.**
- C. The agent has made more than a defined number of tool calls without completing.
- D. The user has sent more than three messages without being satisfied with the response.

Correct Answer: B

Explanation

The clearest indicator for escalation is an irreversible action under uncertainty — the agent needs to do something that cannot be undone and is not confident it is the right action. This is exactly when human judgment must be brought in before proceeding.

Q75. Menler's AI Agents & Workflows bank targets practitioners at what level?

- A. Researchers studying multi-agent coordination and emergent AI behaviour.
- B. Beginners learning what AI agents are for the first time.
- C. Executives deciding whether to invest in agentic AI for their organisation.
- D. Those designing and building production AI agent systems with real reliability requirements.**

Correct Answer: D

Explanation

The AI Agents & Workflows bank tests the knowledge needed to design, build, and operate production agent systems — tool definitions, scaffolding, error handling, multi-agent coordination, and safety. It is for practitioners building real systems, not researchers studying agents theoretically or beginners exploring the concept.



Your turning point in the AI era.

END OF QUESTION BANK

*Your turning
point starts
here.*